



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

EMBEDDED C/C++ PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A IoT & Edge

TOTAL: 10 QUESTIONS

Q1 What is Embedded C/C++ and what problem in IoT & Edge does it solve?

Q6 How does Embedded C/C++ handle reliability and high availability?

Q2 What are the core components or concepts in Embedded C/C++?

Q7 What observability does Embedded C/C++ provide out of the box?

Q3 How does Embedded C/C++ compare to its main alternatives?

Q8 What are common performance bottlenecks in Embedded C/C++?

Q4 How do you get started with Embedded C/C++ for a new project?

Q9 What security best practices apply when running Embedded C/C++?

Q5 What configuration options most affect Embedded C/C++ behavior in production?

Q10 What mistakes do teams commonly make when adopting Embedded C/C++?



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Embedded C/C++ is a tool or platform

Mitigation

Embedded C/C++ is a tool or platform that automates or simplifies a specific concern in the IoT & Edge domain, reducing manual effort and operational complexity for engineering teams.

A6 Embedded C/C++ provides HA through leader election, replication, or stateless

High Availability

Embedded C/C++ provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A2 Embedded C/C++ has key building blocks that

Primitives

Embedded C/C++ has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A7 Embedded C/C++ exposes metrics (Prometheus endpoint or built-in dashboard),

Monitoring

Embedded C/C++ exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A3 Embedded C/C++ trades off simplicity against feature richness differently than its

Hardening

alternatives. Choose Embedded C/C++ when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A8 Performance bottlenecks typically stem from misconfigured connection pools,

Performance

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A4 Install Embedded C/C++ via its package manager

Gotchas

Install Embedded C/C++ via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A9 Run Embedded C/C++ with the principle of least privilege

Assessment

Run Embedded C/C++ with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A5 Production-critical settings typically include concurrency limits, timeout values,

Configuration

retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A10 Common pitfalls include skipping the documentation and learning the API by trial

Documentation

and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.